

UNIT – 4

Graph Algorithms

Detailed Notes

1. Fundamentals of Graph

A graph $G = (V, E)$ consists of a set of vertices V (also called nodes) and a set of edges E connecting pairs of vertices.

Types of Graphs

Graph Type	Description
Undirected Graph	Edges have no direction. If (u,v) is an edge, v is adjacent to u .
Directed Graph (Digraph)	Edges have direction. Edge (u,v) goes from u to v only.
Weighted Graph	Each edge has a weight/cost assigned to it.
Unweighted Graph	All edges have equal weight (or no weight).
Cyclic Graph	Contains at least one cycle (path starting and ending at same vertex).
Acyclic Graph	No cycles. DAG = Directed Acyclic Graph.
Complete Graph	Every pair of distinct vertices is connected by a unique edge.
Connected Graph	There is a path between every pair of vertices.
Tree	A connected, acyclic graph with n vertices and $n-1$ edges.

Key Terms

- **Vertex (Node):** A fundamental unit represented by a point.
- **Edge:** A connection between two vertices.
- **Degree:** Number of edges incident to a vertex. In directed graphs: in-degree and out-degree.
- **Path:** A sequence of vertices where each adjacent pair is connected by an edge.
- **Cycle:** A path that starts and ends at the same vertex.
- **Neighbour:** Vertices connected by an edge to a given vertex.

- **Self-loop:** An edge from a vertex to itself.
- **Multi-edge:** Multiple edges between the same pair of vertices.

2. Representation of Graphs

Adjacency Matrix

A 2D array of size $V \times V$ where $\text{matrix}[i][j] = 1$ if an edge exists between vertices i and j , else 0. For weighted graphs, $\text{matrix}[i][j] = \text{weight}$, or ∞ if no edge.

<code>matrix[i][j] = 1</code> (edge exists)		<code>matrix[i][j] = 0</code> (no edge)
---	--	---

Example — Undirected Graph (4 vertices)

```

    0  1  2  3
  +-----+
0 | 0  1  0  1
1 | 1  0  1  0
2 | 0  1  0  1
3 | 1  0  1  0

```

`Matrix[0][1] = 1` → edge exists between vertex 0 and vertex 1

Python Implementation

```

class GraphMatrix:
    def __init__(self, vertices):
        self.V = vertices
        self.matrix = [[0] * vertices for _ in range(vertices)]

    def add_edge(self, u, v, weight=1):
        self.matrix[u][v] = weight
        self.matrix[v][u] = weight # undirected

    def display(self):
        for row in self.matrix:
            print(row)

```

Adjacency Matrix — Complexity

- | | |
|--|----------|
| ▶ Adding edge: | $O(1)$ |
| ▶ Checking edge: | $O(1)$ |
| ▶ Finding all neighbours: | $O(V)$ |
| ▶ Space Complexity: | $O(V^2)$ |
| ▶ Best for: Dense graphs (many edges), fast edge lookup, small V | |

Adjacency List

Each vertex maintains a list of its neighbouring vertices. Much more space-efficient for sparse graphs.

Example

```
0 -> [1, 3]
1 -> [0, 2]
2 -> [1, 3]
3 -> [0, 2]
```

Python Implementation

```
from collections import defaultdict

class GraphList:
    def __init__(self, vertices):
        self.V = vertices
        self.graph = defaultdict(list)

    def add_edge(self, u, v, weight=1):
        self.graph[u].append((v, weight))
        self.graph[v].append((u, weight)) # undirected

    def display(self):
        for v in self.graph:
            print(f"{v} -> {self.graph[v]}")
```

Adjacency List — Complexity

- ▶ Adding edge: $O(1)$
- ▶ Checking edge: $O(\text{degree}) = O(V)$ worst case
- ▶ Finding all neighbours: $O(\text{degree}) = O(V)$ worst case
- ▶ Space Complexity: $O(V + E)$
- ▶ Best for: Sparse graphs (few edges), iterating neighbours efficiently

Comparison: Matrix vs List

Criteria	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Check Edge	$O(1)$	$O(\text{degree})$
Iterate Neighbours	$O(V)$	$O(\text{degree})$
Best For	Dense graphs	Sparse graphs

3. Path Matrix (Reachability Matrix)

A path matrix P (or reachability matrix) of size $V \times V$ where $P[i][j] = 1$ if there exists a path from vertex i to vertex j , else 0.

The path matrix can be computed using the Floyd-Warshall algorithm by repeatedly updating reachability through intermediate vertices.

Floyd-Warshall Algorithm for Path Matrix

```
def path_matrix(graph, V):
    P = [[0]*V for _ in range(V)]

    # Initialize with direct edges
    for i in range(V):
        for j in range(V):
            if graph[i][j]:
                P[i][j] = 1

    # Find transitive closure (all reachable pairs)
    for k in range(V): # intermediate vertex
        for i in range(V): # source
            for j in range(V): # destination
                P[i][j] = P[i][j] or (P[i][k] and P[k][j])

    return P

# Time:  $O(V^3)$     Space:  $O(V^2)$ 
```

4. Breadth First Search (BFS)

BFS explores a graph level by level, visiting all neighbours before moving to the next level. It uses a Queue (FIFO).

Algorithm

```
BFS(G, start):
    visited[] = false for all vertices
    queue = empty
    visited[start] = true
    queue.enqueue(start)

    while queue is not empty:
        v = queue.dequeue()
        visit(v) # process vertex v
        for each neighbour u of v:
            if not visited[u]:
                visited[u] = true
                queue.enqueue(u)
```

Properties

BFS Properties

- ▶ Time Complexity: $O(V + E)$
- ▶ Space Complexity: $O(V)$
- ▶ Produces shortest path in unweighted graphs
- ▶ Level-order traversal
- ▶ Uses Queue (FIFO) data structure

Example

Graph: 0-1-2, 0-3, 1-4, 2-4, 3-4

```
BFS from vertex 0:  
Level 0: 0  
Level 1: 1, 3  
Level 2: 2, 4  
Order: 0 -> 1 -> 3 -> 2 -> 4
```

Complete Python Implementation

```
from collections import deque  
  
def bfs(graph, start):  
    visited = set()  
    queue = deque([start])  
    visited.add(start)  
    result = []  
  
    while queue:  
        vertex = queue.popleft()  
        result.append(vertex)  
        for neighbour in graph[vertex]:  
            if neighbour not in visited:  
                visited.add(neighbour)  
                queue.append(neighbour)  
    return result  
  
graph = {0:[1,3], 1:[0,2,4], 2:[1,4], 3:[0,4], 4:[1,2,3]}  
print(bfs(graph, 0)) # Output: [0, 1, 3, 2, 4]
```

5. Depth First Search (DFS)

DFS explores a graph as deep as possible along each branch before backtracking. It uses a Stack (LIFO) or recursion.

Algorithm

```
DFS(G, start):
```

```

visited[] = false for all vertices
stack = empty
stack.push(start)

while stack is not empty:
    v = stack.pop()
    if not visited[v]:
        visited[v] = true
        visit(v)                # process vertex v
        for each neighbour u of v (reverse order):
            if not visited[u]:
                stack.push(u)

```

Properties

DFS Properties

- ▶ Time Complexity: $O(V + E)$
- ▶ Space Complexity: $O(V)$ (recursion stack or explicit stack)
- ▶ Backtracking-based exploration
- ▶ Useful for cycle detection, path finding, topological sort
- ▶ Uses Stack (LIFO) data structure (or recursion)

Example

Graph: 0-1-2, 0-3, 1-4, 2-4, 3-4

```

DFS from vertex 0 (using stack):
Visit 0, push neighbours: push 1, 3
Pop 3, push its neighbours: 4
Pop 4, push its neighbours: 2, 1 (already visited)
Pop 2, push its neighbours: (none new)
Order: 0 -> 3 -> 4 -> 2 -> 1

```

Complete Python Implementation

```

# Iterative DFS
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    result = []
    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            result.append(vertex)
            for neighbour in reversed(graph[vertex]):
                if neighbour not in visited:
                    stack.append(neighbour)
    return result

# Recursive DFS

```

```
def dfs_recursive(graph, vertex, visited, result):
    visited.add(vertex)
    result.append(vertex)
    for neighbour in graph[vertex]:
        if neighbour not in visited:
            dfs_recursive(graph, neighbour, visited, result)

graph = {0:[1,3], 1:[0,2,4], 2:[1,4], 3:[0,4], 4:[1,2,3]}
print(dfs_iterative(graph, 0))    # Output: [0, 1, 2, 4, 3]
```

6. Spanning Trees

A spanning tree of a graph G is a subgraph that is a tree and contains all vertices of G .

Spanning Tree: V vertices | Exactly $V-1$ edges | Connected & Acyclic

Properties

Spanning Tree Properties

- ▶ Contains ALL V vertices of the original graph
- ▶ Has exactly $V-1$ edges
- ▶ Is a connected, acyclic graph
- ▶ Every connected graph has at least one spanning tree
- ▶ Can be found using BFS or DFS traversal

Spanning Tree from BFS / DFS

When traversing a graph with BFS or DFS, the edges used to discover new vertices (tree edges) form a spanning tree. Tree edges connect a vertex to a previously undiscovered vertex.

7. Minimum Spanning Trees (MST)

A Minimum Spanning Tree in a weighted, connected graph is a spanning tree with the minimum total edge weight.

Kruskal's Algorithm

A greedy algorithm that sorts all edges by weight and adds the smallest edge that does not form a cycle.

Kruskal(G) :

```

    sort all edges by weight (ascending)
    for each edge (u, v) in sorted order:
        if u and v are in different components:
            add edge (u, v) to MST
            union the two components

```

Time Complexity: $O(E \log E)$ or $O(E \log V)$
 Space: $O(V + E)$

Prim's Algorithm

A greedy algorithm that grows the MST from a starting vertex, always adding the minimum weight edge connecting a visited vertex to an unvisited vertex.

```

Prim(G, start):
    visited[start] = true
    result = empty
    while not all vertices visited:
        find minimum weight edge (u, v)
            where u is visited, v is not
        add edge (u, v) to result
        mark v as visited

```

Time Complexity: $O(E \log V)$ with priority queue
 Space: $O(V + E)$

Example

Graph: 4 vertices, edges: 0-1(10), 0-2(6), 0-3(5), 1-2(3), 2-3(4)

```

Kruskal's MST:
Step 1: Pick 1-2 (weight 3) → add
Step 2: Pick 2-3 (weight 4) → add
Step 3: Pick 0-3 (weight 5) → add
MST edges: (1,2), (2,3), (0,3)
Total weight: 3 + 4 + 5 = 12

```

8. Dijkstra's Shortest Path Algorithm

Dijkstra's algorithm finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

Algorithm

```

Dijkstra(G, source):
    dist[] = infinity for all vertices
    dist[source] = 0
    visited[] = false for all vertices
    priority_queue = min-heap with (distance, vertex)

    while priority_queue is not empty:

```



```

    (d, u) = extract_min(priority_queue)
    if visited[u]: continue
    visited[u] = true
    for each neighbour v of u with weight w:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            add (dist[v], v) to priority_queue

    return dist[]

Time:  $O((V + E) \log V)$  with priority queue
Space:  $O(V)$ 

```

Example

Graph: $0 \rightarrow 1(4)$, $0 \rightarrow 2(2)$, $1 \rightarrow 2(1)$, $1 \rightarrow 3(5)$, $2 \rightarrow 3(8)$, $2 \rightarrow 4(10)$, $3 \rightarrow 4(2)$

```

Dijkstra from source 0:
Initial:          dist = [0, ∞, ∞, ∞, ∞]
After processing 0: dist = [0, 4, 2, ∞, ∞]
After processing 2: dist = [0, 3, 2, 10, 12]
After processing 1: dist = [0, 3, 2, 8, 12]
After processing 3: dist = [0, 3, 2, 8, 10]

Shortest paths from 0:
0→0 = 0    0→2 = 2    0→1 = 3    0→3 = 8    0→4 = 10

```

Important Notes — Dijkstra

- Does NOT work with negative edge weights
- Greedy algorithm — makes locally optimal choice at each step
- Produces shortest paths in weighted graphs with non-negative weights
- Equivalent to BFS when all edge weights are equal

9. Bellman-Ford Algorithm

The Bellman-Ford algorithm finds shortest paths from a source to all vertices, handling graphs with negative edge weights. It can also detect negative weight cycles.

Algorithm

```

Bellman-Ford(G, source):
    dist[] = infinity for all vertices
    dist[source] = 0

    # Relax all edges V-1 times
    repeat V-1 times:
        for each edge (u, v) with weight w:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

```

```
# Check for negative weight cycles
for each edge (u, v) with weight w:
    if dist[u] + w < dist[v]:
        return "Negative cycle detected"

return dist[]

Time Complexity: O(V × E)
Space Complexity: O(V)
```

Why V-1 Iterations?

In the worst case, the shortest path between any two vertices can have at most $V-1$ edges (simple path without cycles). Each iteration finds shortest paths with at most one more edge than before.

Example

Graph: $0 \rightarrow 1(4)$, $0 \rightarrow 2(2)$, $1 \rightarrow 2(-3)$, $1 \rightarrow 3(5)$, $2 \rightarrow 3(2)$

```
Bellman-Ford from source 0:
Initial:      dist = [0, ∞, ∞, ∞]
After iteration 1: dist = [0, 4, 2, 4]
After iteration 2: dist = [0, 1, 2, 4]    (negative edge improved path)
No further improvement → no negative cycle

Shortest paths: 0→0=0, 0→2=2, 0→1=1 (via 2), 0→3=4 (via 2)
```

Comparison: Dijkstra vs Bellman-Ford

Criteria	Dijkstra	Bellman-Ford
Time Complexity	$O((V+E) \log V)$	$O(V \times E)$
Edge Weights	Non-negative only	Can be negative
Negative Cycle	Not detected	Detected
Algorithm Type	Greedy	Dynamic Programming

10. Summary & Quick Reference

Algorithm	Time Complexity	Space	Use Case
BFS	$O(V + E)$	$O(V)$	Shortest path (unweighted)
DFS	$O(V + E)$	$O(V)$	Cycle detection, path finding
Kruskal (MST)	$O(E \log E)$	$O(V + E)$	Minimum spanning tree
Prim (MST)	$O(E \log V)$	$O(V + E)$	Minimum spanning tree

Dijkstra	$O((V+E) \log V)$	$O(V)$	Shortest path (non-neg weights)
Bellman-Ford	$O(V \times E)$	$O(V)$	Shortest path (any weights)
Floyd-Warshall	$O(V^3)$	$O(V^2)$	All-pairs shortest path